

Esercizio 2 — Server Linux

Autore: Francesco Terracciano · **Data:** 7 settembre 2026 · **Ambiente:** Kali Linux (VM su UTM) · **Utente:** kali

Obiettivo del laboratorio: usare la riga di comando Linux per identificare i server in esecuzione sulla macchina, collegare ogni porta al processo responsabile e verificare la reale natura dei servizi. Di seguito le operazioni svolte, nell'ordine in cui sono state eseguite, con gli output raccolti e le risposte alle domande della traccia.

1 Avvio di nginx e visualizzazione dei processi

Ho avviato il web server nginx con privilegi elevati e ho poi elencato i processi in esecuzione, prima in formato lungo con `sudo ps -elf` e successivamente come albero gerarchico con `sudo ps -ejH`. L'uso di `sudo` è stato necessario per poter vedere anche i processi appartenenti a root e agli altri utenti.

```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo /usr/sbin/nginx
[sudo] password for kali:
(kali@kali)-[~]
$ sudo ps -elf
sudo ps -ejH
F S UID          PID    PPID    C  PRI   NI     ADDR  SZ  WCHAN    STIME  TTY          STATE TIME  CMD
4 S root           0      0      0   80   0    -     0    0  do_epo Sep06 ?          T 00:00:03 /sbin/init splash
1 S root           1      0      0   80   0    -     0    0  kthrea Sep06 ?          T 00:00:00 [kthreadd]
1 S root           2      0      0   80   0    -     0    0  kthrea Sep06 ?          T 00:00:00 [pool_workqueue_releas
1 I root          34      0      0   60  -20   -     0    0  rescue Sep06 ?          E 00:00:00 [kworker/R-rcu_gp]
1 I root          5      0      0   60  -20   -     0    0  rescue Sep06 ?          T 00:00:00 [kworker/R-sync_wq]
1 I root           6      0      0   60  -20   -     0    0  rescue Sep06 ?          T 00:00:00 [kworker/R-kvfree_rcu_
1 I root           7      0      0   60  -20   -     0    0  rescue Sep06 ?          E 00:00:00 [kworker/R-slub_flushw
1 I root           8      0      0   60  -20   -     0    0  rescue Sep06 ?          E 00:00:00 [kworker/R-netns]
1 I root          10      0      0   60  -20   -     0    0  worker Sep06 ?          E 00:00:00 [kworker/0:0H-kblockd]
1 I root          13      0      0   60  -20   -     0    0  rescue Sep06 ?          E 00:00:00 [kworker/R-mm_percpu_w
1 S root          14      0      0   80   0    -     0    0  smpboo Sep06 ?          E 00:00:01 [ksoftirqd/0]
1 I root          15      0      0   80   0    -     0    0  rcu_gp Sep06 ?          T 00:00:15 [rcu_sched]
1 S root          16      0      0   80   0    -     0    0  kthrea Sep06 ?          T 00:00:00 [rcu_exp_par_gp_kthrea
1 S root          17      0      0   80   0    -     0    0  kthrea Sep06 ?          T 00:00:00 [rcu_exp_gp_kthread_wd
1 S root          18      0      0  -40   -    -     0    0  smpboo Sep06 ?          T 00:00:00 [migration/0]
1 S root          19      0      0   80   0    -     0    0  smpboo Sep06 ?          T 00:00:00 [cpuhp/0]
1 S root          20      0      0   80   0    -     0    0  smpboo Sep06 ?          T 00:00:00 [cpuhp/1]
1 S root          21      0      0  -40   -    -     0    0  smpboo Sep06 ?          T 00:00:00 [migration/1]
1 S root          22      0      0   80   0    -     0    0  smpboo Sep06 ?          T 00:00:01 [ksoftirqd/1]
1 I root          24      0      0   60  -20   -     0    0  worker Sep06 ?          T 00:00:00 [kworker/1:0H-kblockd]
1 S root          25      0      0   80   0    -     0    0  smpboo Sep06 ?          T 00:00:00 [cpuhp/2]
1 S root          26      0      0  -40   -    -     0    0  smpboo Sep06 ?          T 00:00:00 [migration/2]
1 S root          27      0      0   80   0    -     0    0  smpboo Sep06 ?          T 00:00:01 [ksoftirqd/2]
1 I root          29      0      0   60  -20   -     0    0  worker Sep06 ?          T 00:00:00 [kworker/2:0H-kblockd]
1 S root          30      0      0   80   0    -     0    0  smpboo Sep06 ?          T 00:00:00 [cpuhp/3]
1 S root          31      0      0  -40   -    -     0    0  smpboo Sep06 ?          T 00:00:00 [migration/3]
1 S root          32      0      0   80   0    -     0    0  smpboo Sep06 ?          T 00:00:01 [ksoftirqd/3]

```

Fig. 1 — Avvio di `sudo /usr/sbin/nginx` e output di `sudo ps -elf`.

Perché è stato necessario eseguire ps come root (con sudo)?

Senza privilegi elevati `ps` mostra solo i processi del proprio utente. Per elencare quelli di root e degli altri utenti — come il master nginx (root) e i suoi worker — servono i privilegi di root ottenuti con `sudo`.

2 Individuazione del master, dei worker e della gerarchia

Nell'output ho individuato il processo **master** di nginx (PID `3310475`, utente **root**, avviato dal processo `1` di **systemd**) e i quattro processi **worker** (PID `3310476` – `3310479`, utente **www-data**), tutti con PPID `3310475`. La vista ad albero di `ps -ejH` conferma la relazione padre-figlio tramite l'indentazione.

```
1 I root      3301891      2 0 80 0 - 0 worker 08:25 ? 00:00:00 [kworker/2:1-events]
1 I root      3304448      2 0 80 0 - 0 worker 08:30 ? 00:00:00 [kworker/0:2-mm_percp
1 I root      3304772      2 0 80 0 - 0 worker 08:31 ? 00:00:00 [kworker/2:2-events]
1 I root      3304869      2 0 80 0 - 0 worker 08:31 ? 00:00:00 [kworker/3:1-events]
1 I root      3307327      2 0 80 0 - 0 worker 08:36 ? 00:00:00 [kworker/1:0-events]
1 I root      3307453      2 0 80 0 - 0 worker 08:36 ? 00:00:00 [kworker/2:0-events]
1 I root      3307630      2 0 80 0 - 0 worker 08:36 ? 00:00:00 [kworker/3:2-events]
1 I root      3307761      2 0 80 0 - 0 worker 08:37 ? 00:00:00 [kworker/0:1-events]
4 S root      3308501     359 0 80 0 - 2349 skb_wa 08:38 ? 00:00:00 systemd-userwork: wa
0 S kali      3308802      1 0 80 0 - 205068 poll_s 08:39 ? 00:00:00 /usr/bin/qterminal
0 S kali      3308810 3308802 0 80 0 - 2758 sigsus 08:39 pts/0 00:00:00 /usr/bin/zsh
1 I root      3308953      2 0 80 0 - 0 worker 08:39 ? 00:00:00 [kworker/u16:2-events
1 I root      3308954      2 0 80 0 - 0 worker 08:39 ? 00:00:00 [kworker/u16:4]
4 S root      3310272     359 0 80 0 - 2314 skb_wa 08:41 ? 00:00:00 systemd-userwork: wa
1 S root      3310475      1 0 80 0 - 3714 sigsus 08:41 ? 00:00:00 nginx: master process
5 S www-data 3310476 3310475 0 80 0 - 4118 do_epo 08:41 ? 00:00:00 nginx: worker process
5 S www-data 3310477 3310475 0 80 0 - 4118 do_epo 08:41 ? 00:00:00 nginx: worker process
5 S www-data 3310478 3310475 0 80 0 - 4118 do_epo 08:41 ? 00:00:00 nginx: worker process
5 S www-data 3310479 3310475 0 80 0 - 4118 do_epo 08:41 ? 00:00:00 nginx: worker process
4 S root      3310480     359 0 80 0 - 2314 skb_wa 08:41 ? 00:00:00 systemd-userwork: wa
4 S root      3310560 3308810 0 80 0 - 5290 poll_s 08:41 pts/0 00:00:00 sudo ps -elf
1 S root      3310562 3310560 0 80 0 - 5290 poll_s 08:41 pts/1 00:00:00 sudo ps -elf
4 R root      3310563 3310562 0 80 0 - 2328 - 08:41 pts/1 00:00:00 ps -elf
  PID      PGID      SID TTY          TIME CMD
    2        0        0 ?          00:00:00 kthreadd
    3        0        0 ?          00:00:00 pool_workqueue_release
    4        0        0 ?          00:00:00 kworker/R-rcu_gp
    5        0        0 ?          00:00:00 kworker/R-sync_wq
    6        0        0 ?          00:00:00 kworker/R-kvfree_rcu_reclaim
    7        0        0 ?          00:00:00 kworker/R-slub_flushwq
    8        0        0 ?          00:00:00 kworker/R-netns
   10        0        0 ?          00:00:00 kworker/0:0H-kblockd
   13        0        0 ?          00:00:00 kworker/R-mm_percpu_wq
   14        0        0 ?          00:00:01 ksoftirqd/0
   15        0        0 ?          00:00:15 rcu_sched
```

Fig. 2 — Master nginx (`3310475`, root) e worker (`3310476`-`3310479`, www-data); in basso l'albero di `ps -ejH`.

Come viene rappresentata la gerarchia dei processi da ps?

Con `ps -ejH` la gerarchia è resa tramite l'indentazione: i figli appaiono rientrati sotto il padre. La relazione è confermata dalla colonna PPID: i worker hanno tutti PPID `3310475`, cioè discendono dal master.

Un processo che ne genera altri è un comportamento comune?

Sì. È il classico modello master/worker: un processo principale legge la configurazione e gestisce processi figli che svolgono il lavoro effettivo. È tipico dei server (nginx, Apache, database) e garantisce scalabilità e isolamento dei guasti.

3 Elenco delle porte in ascolto con netstat

Ho usato `sudo netstat -tunap` per visualizzare le connessioni di rete e i servizi in ascolto. L'output mostra la porta 80 in stato LISTEN associata al processo nginx, sia su IPv4 (`0.0.0.0:80`) sia su IPv6 (`:::80`).

```
(kali@kali)-[~]
$ sudo netstat -tunap
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:80             0.0.0.0:*               LISTEN      3310475/nginx: mast
tcp        0      0 127.0.0.1:11434        0.0.0.0:*               LISTEN      518/ollama
tcp        0      0 192.168.64.9:37026     15.197.213.252:443     ESTABLISHED 765/expressvpn-daem
tcp        0      0 100.64.100.6:33402     104.18.38.107:443     ESTABLISHED 765/expressvpn-daem
tcp6       0      0 :::80                  :::*                    LISTEN      3310475/nginx: mast
udp        0      0 192.168.64.9:59862     64.6.64.6:53           ESTABLISHED 765/expressvpn-daem
udp        0      0 192.168.64.9:36319     64.6.64.6:53           ESTABLISHED 765/expressvpn-daem
udp        0      0 192.168.64.9:68        192.168.64.1:67        ESTABLISHED 615/NetworkManager
udp        0      0 192.168.64.9:47321     64.6.64.6:53           ESTABLISHED 765/expressvpn-daem
udp        0      0 192.168.64.9:57660     64.6.64.6:53           ESTABLISHED 765/expressvpn-daem
udp6       0      0 fe80::1838:b37:dc73:546 :::*                    ESTABLISHED 615/NetworkManager
```

Fig. 3 — `sudo netstat -tunap` : porta 80 in LISTEN associata al PID 3310475/nginx .

Qual è il significato delle opzioni -t, -u, -n, -a e -p in netstat?

-t mostra le connessioni TCP; -u quelle UDP; -n mostra indirizzi e porte in forma numerica (senza risoluzione DNS); -a mostra tutte le socket (in ascolto e stabilite); -p aggiunge PID e nome del programma associato.

L'ordine delle opzioni è importante per netstat?

No. Le opzioni possono essere raggruppate sotto un unico - e l'ordine non cambia il risultato.

Per la porta 80: protocollo di Livello 4, stato della connessione e PID del processo?

Protocollo L4 = **TCP**; stato = **LISTEN**; PID = **3310475** (nginx). La porta risulta in ascolto sia su IPv4 sia su IPv6.

Che tipo di servizio è in esecuzione sulla porta 80 TCP?

Per convenzione la porta 80 TCP è assegnata al servizio HTTP (web): si può quindi ipotizzare la presenza di un web server, ipotesi poi verificata con il telnet.

4 Incrocio tra netstat e ps

Per legare in modo univoco la porta al processo ho filtrato l'output di `ps` sul PID di nginx con `sudo ps -elf | grep 3310475`. Il risultato mostra il master (root, avviato dal processo 1) e i quattro worker (www-data) che ne discendono.

```
(kali@kali)-[~]
$ sudo ps -elf | grep 3310475
1 S root      3310475      1 0 80    0 - 3714 sigsus 08:41 ?        00:00:00 nginx: master process /usr/sbin/nginx
5 S www-data 3310476 3310475 0 80    0 - 4118 do_epo 08:41 ?        00:00:00 nginx: worker process
5 S www-data 3310477 3310475 0 80    0 - 4118 do_epo 08:41 ?        00:00:00 nginx: worker process
5 S www-data 3310478 3310475 0 80    0 - 4118 do_epo 08:41 ?        00:00:00 nginx: worker process
5 S www-data 3310479 3310475 0 80    0 - 4118 do_epo 08:41 ?        00:00:00 nginx: worker process
0 S kali      3329168 3308810 0 80    0 - 1568 anon_p 09:17 pts/0    00:00:00 grep --color=auto 3310475
```

Fig. 4 — `sudo ps -elf | grep 3310475` : master e worker nginx associati al PID individuato nel netstat.

Come si può concludere che il PID 3310475 è nginx?

Incrociando gli output: `netstat` associa la porta 80 al PID 3310475 con nome `nginx: master`, e `ps -elf | grep 3310475` mostra la riga `nginx: master process /usr/sbin/nginx`. La coincidenza del PID tra i due comandi lega la porta al processo senza ambiguità.

Perché l'ultima riga dell'output mostra "grep 3310475"?

Perché il comando `grep` compare tra i processi attivi nell'istante in cui `ps` scatta la fotografia: i due comandi girano in parallelo nella stessa pipeline, quindi `grep` "trova anche se stesso". È rumore normale e non fa parte di `nginx`.

Cos'è nginx? Qual è la sua funzione?

`nginx` è un web server open source che resta in ascolto su una porta (80 per HTTP, 443 per HTTPS) e risponde alle richieste dei client servendo contenuti web. È usato anche come reverse proxy, load balancer e per la terminazione TLS. La sua architettura asincrona a eventi (un master + più worker) gli permette di gestire moltissime connessioni con poche risorse — coerentemente con l'output osservato.

Quali sono i vantaggi dell'uso di netstat?

Con un solo comando fornisce la fotografia completa dello stato di rete: quali servizi sono in ascolto (superficie d'attacco), quali connessioni sono attive e verso chi, e — con `-p` — quale processo è responsabile di ogni porta. È quindi rapido per fare inventory dei servizi e per attività di diagnosi e investigazione. (Nota: su Linux è oggi deprecato in favore di `ss`, più veloce.)

5 Interrogazione del servizio sulla porta 80 con Telnet

Per verificare la reale natura del servizio ho stabilito una connessione TCP con `telnet 127.0.0.1 80` e ho inviato una stringa non valida (`ciao`). `nginx` ha risposto con un errore `HTTP/1.1 400 Bad Request` in formato pagina HTML, rivelando nel banner `Server: nginx`.

```
udp        0      0 192.168.64.9:57660    64.6.64.6:53
udp6      0      0 fe80::1838:b37:dc73:546 :::*

(kali@kali)-[~]
$ telnet 127.0.0.1 80
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
ciao
HTTP/1.1 400 Bad Request
Server: nginx
Date: Mon, 07 Sep 2026 12:56:07 GMT
Content-Type: text/html
Content-Length: 150
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx</center>
</body>
</html>
Connection closed by foreign host.

(kali@kali)-[~]
```

Fig. 5 — `telnet 127.0.0.1 80` : risposta 400 Bad Request con banner `Server: nginx`.

Perché l'errore è stato inviato come pagina web?

Perché `nginx` è un web server: alla stringa non valida non corrisponde una richiesta HTTP corretta, quindi risponde nell'unico linguaggio che conosce — una risposta HTTP con corpo HTML. Il fatto stesso che risponda in HTTP conferma che si tratta di un web server.

Se un attaccante rinominasse un malware in "nginx", come potrebbe un analista esserne sicuro?

Non fidandosi del nome del processo, ma interrogando il servizio: un vero nginx, contattato via telnet, risponde con un messaggio HTTP che ne rivela identità e versione. È il comportamento a Livello 7, non il nome, a costituire la prova. In aggiunta si potrebbero verificare il percorso del binario, il suo hash e la firma del pacchetto.

Quali sono i vantaggi dell'uso di Telnet?

Come strumento diagnostico: verifica in un istante se una porta TCP è raggiungibile, permette il banner grabbing per identificare servizio e versione, e consente di dialogare a mano con protocolli testuali. Va però ricordato che come protocollo di accesso remoto è insicuro, perché trasmette tutto in chiaro: per amministrare i sistemi si usa SSH, che offre le stesse funzioni in forma cifrata.

6 Verifica di altre porte TCP con Telnet

Ho infine testato altre porte con telnet. Le porte 22, 23, 395 e 68 hanno restituito **Connection refused**: la macchina risponde attivamente ma su quelle porte non è in ascolto alcun servizio TCP. Il risultato contrasta con la porta 80, dove la connessione si era stabilita e nginx aveva risposto.

```
(kali㉿kali)-[~]
$ telnet 127.0.0.1 22
Trying 127.0.0.1 ...
telnet: Unable to connect to remote host: Connection refused

(kali㉿kali)-[~]
$ telnet 127.0.0.1 23
Trying 127.0.0.1 ...
telnet: Unable to connect to remote host: Connection refused

(kali㉿kali)-[~]
$ telnet 127.0.0.1 395
Trying 127.0.0.1 ...
telnet: Unable to connect to remote host: Connection refused

(kali㉿kali)-[~]
$ telnet 127.0.0.1 68
Trying 127.0.0.1 ...
telnet: Unable to connect to remote host: Connection refused

(kali㉿kali)-[~]
```

Fig. 6 — telnet verso le porte 22, 23, 395 e 68: tutte **Connection refused**.

Cosa succede connettendosi con Telnet alla porta 68?

Il telnet fallisce con Connection refused, perché la porta 68 è usata dal servizio DHCP, che opera su UDP, mentre telnet lavora solo su TCP: non c'è alcun servizio TCP in ascolto da contattare. Lo conferma il netstat, dove la 68 compare come **udp**.

Esito. Il servizio in ascolto sulla porta 80 è stato identificato come il web server nginx (PID 3310475), collegando la porta al processo tramite l'incrocio di **netstat** e **ps** e confermandone la natura tramite il banner ottenuto con telnet. Le altre porte testate non risultano associate ad alcun servizio TCP.